

Mock 1 · Part C — CV Deep Dive (40 min): Full Answer Guide

Tracker ID: FN-25 · Generated: 2026-09-29 · Source:

[Komol_CV_FieldNation_Software_Engineer.docx](#), [_memory/story-bank.md](#) (S1–S9),
[mock-01-cv-behavioral.md](#) Part C Language: **Full English** (Mock-series exception, decision D8)
 Parent: [mock-01-cv-behavioral.md](#) → this file expands Part C only.

How to use this doc

Every block (C1–C10) has the same shape:

Section	What it gives you
Opening + model answer	The first question and a 90–120 s STAR answer
Drill questions	For each follow-up: <i>why they ask</i> → a strong answer (45–90 s spoken) → <i>if they push further</i>
Trade-off line	1 main line + 2 alternatives, so you don't sound memorized
Numbers to know	Every number, where it comes from, and how to defend it
Key phrases	The phrase, when to use it, and an example sentence

The honesty rule (read this first)

Some answers depend on facts only you know (the real pain, how a number was measured, which gateway). This doc **never invents them**. Instead you'll see:

 **Pick your real version:** Option A / B / C — each is a full answer. **Say only the one that is true.**
 Delete the others when you fill this in.

[**FILL:** ...] = a small detail only you can add (a name, a number, a time period).

The 4-step formula for any drill question

US interviewers like answers that start with the conclusion. Use this order:

1. **Direct answer** (1 sentence) — "We used a shared database during the transition."
2. **How** (2–3 sentences) — the mechanism.
3. **Proof / example** (1 sentence) — a number, a real case.
4. **Trade-off or lesson** (1 sentence) — "The downside was..." / "Next time I'd..."

Safe phrases when you don't know or don't remember

- "I don't remember the exact number, but it was roughly [**range**], and here's how we measured it..."
- "I wasn't directly responsible for that part — [**teammate role**] owned it — but my understanding is..."

- "We didn't do that, and honestly it's something I'd add today. Here's how I'd approach it..."
- "Good question. Let me think for a second." (*Pausing is fine. Silence for 3 seconds is better than a wrong answer.*)

C1 Monolith → Event-driven Microservices (S1)

Opening: "Tell me about the biggest technical project you've led."

Model answer (STAR, ~2 min):

S: "At Sokrio we had a Laravel 8 and Vue.js 2 monolith serving 100+ enterprise clients. [FILL: the real pain — e.g. one heavy report could slow down the whole app / every small change needed a full deploy / the frontend and backend were tightly coupled.] **T:** I was asked to lead the migration to a more scalable architecture — **without any downtime**, because clients use the system every day for orders and field work. **A:** We split it into a Laravel core REST API, two separate React apps — one for tenants, one for admin — and three services: **Report, Notification and SOP**, talking over **RabbitMQ**, with MongoDB and Redis. The most important decision was to **run both systems in parallel** and move traffic step by step, so we could roll back at any point. **R:** We cut over with **zero downtime**, and server load dropped by **about 20%**. Just as important, heavy reports stopped affecting the rest of the platform."

 Last sentence of R is only safe if it's true — it depends on your [FILL] pain. If the pain was different, end with the matching benefit.

Drill 1 — "How did you decide the service boundaries?"

Why they ask: To see if you split by **real reasons** or just because microservices are trendy.

Strong answer:

"We split along three questions. **One — does this part scale differently?** Reports are heavy and bursty: at month-end everyone runs them at once. That load shouldn't slow down order taking, so Reports became its own service. **Two — is it naturally asynchronous?** Notifications don't need to happen inside the user's request. The core publishes an event like 'order approved', and the Notification service handles email, SMS or push on its own time. **Three — does it own its own data?** Each service should own its data and not reach into another service's tables. SOP [FILL: one sentence on what SOP does] had its own clear data and rules, so it was a clean cut. Everything else — orders, users, products, tenants — **stayed in the core**, because those parts change together and need strong transactions. We deliberately didn't split too much. Too many small services would have given us network problems without real benefit."

If they push further:

- **"Why not a modular monolith?"** → "That was a real option. We chose separate services for the parts that needed **independent scaling and independent deploys** — mainly reports. For the core, we effectively kept a modular monolith. So it was a hybrid, not microservices everywhere."
- **"What's a bounded context?"** → "A part of the business with its own language and rules — for us, 'reporting' and 'notifications' are different contexts from 'order management'. It's a good guide for where to draw a service line."

Drill 2 — "How did you keep data in sync while both systems were running?"

Why they ask: This is the hardest part of any migration. A vague answer here is a red flag.

Pick your real version:

Option A — Shared database during the transition (most common for Laravel migrations):

"During the transition, the old monolith and the new core API **pointed at the same MySQL databases**. So there was only one source of truth, and nothing to sync. We moved traffic endpoint by endpoint — [FILL: via Nginx routing / the new React apps calling the new API] — and if a new endpoint had a problem, we routed that path back to the old code. The new services like Reports and Notifications **didn't write to the core tables**. They received events over RabbitMQ and built their own data in MongoDB and Redis. Once all traffic was on the new system, we removed the old code."

Option B — Events + separate stores (dual system):

"The core API was the source of truth. Every important change — order created, order approved — was **published as an event to RabbitMQ**. The new services consumed those events and built their own read models in MongoDB. For the historical data, we ran a **one-time backfill** job, then let the events keep it up to date. We compared counts and totals between old and new reports for [FILL: period] before switching users over."

Option C — Per-tenant rollout:

"Because every client has its own tenant database, we could migrate **tenant by tenant**. We moved a few low-risk tenants first, watched errors and performance for [FILL: days/weeks], then moved the rest in batches. If a tenant had a problem, we switched only that tenant back."

If they push further:

- "What if the DB write succeeds but the event publish fails?" → "That's the **dual-write problem**. The clean fix is the **transactional outbox pattern**: write the event into an **outbox** table in the same database transaction, and a separate worker publishes it to RabbitMQ. [FILL: 'We used this' OR 'We didn't have it at first — it's the first thing I'd add today.']"
- "How did you know the new system gave the same results?" → [FILL: compared report outputs / ran both and diffed / QA sign-off from key clients]

Drill 3 — "How did you measure the 20%?"

Why they ask: US interviewers check **every number**. They want: what metric, what period, and was it fair.

Pick your real version:

Option A — CPU / load average:

"We compared **average CPU utilization on the main application servers** in [FILL: Grafana / CloudWatch], over [FILL: two to four] weeks before the cutover and the same length after. We picked comparable periods — not month-end against a quiet week — so traffic was similar. Average CPU dropped by about 20%, mainly because heavy report queries no longer ran on the web servers."

Option B — Requests / response time:

"We looked at [FILL: request throughput per server / p95 response time] in our APM before and after. For the same traffic, the core servers did about 20% less work, because reports and notifications moved to their own services."

Option C — Honest estimate:

"Honestly, it was an estimate from our monitoring dashboards, not a formal benchmark. The trend was clear — about 20% lower load on the core servers — but I wouldn't call it a lab measurement."

If they push further:

- **"Didn't the total infrastructure grow because of the new services?"** → "Yes, that's fair. The 20% is the **core servers**. We added separate machines for the services, but the key win was **isolation**: a heavy report can't take down order taking anymore." [FILL: adjust if total cost also went down]

Drill 4 — "What happens if RabbitMQ goes down, or a message is processed twice?"

Why they ask: To test if you understand **delivery guarantees** — the core of event-driven systems.

Strong answer:

"Two separate problems. **First, a message processed twice.** RabbitMQ gives you **at-least-once delivery**, so duplicates *will* happen — for example, if a consumer crashes after doing the work but before it sends the ack. So **every consumer is idempotent**. Each message has a unique ID, and before processing, the consumer checks if that ID was already handled — [FILL: a processed-messages table with a unique key / a Redis key with a TTL]. If yes, it just acks and skips. For example, a duplicate 'order approved' event won't send the SMS twice. **Second, failures.** Queues are **durable** and messages are **persistent**, so they survive a broker restart. Consumers **ack only after the work is done**. If processing fails, we **retry with a delay**, and after [FILL: 3–5] attempts the message goes to a **dead-letter queue**, where we can look at it and replay it. We alert when the DLQ isn't empty. **If RabbitMQ itself is down**, the core API still works — users can still take orders. The events just wait: [FILL: they're kept in the outbox table / the publisher retries / Laravel's queue retries the publish job]. When the broker is back, they go out."

If they push further:

- **"What about message order?"** → "RabbitMQ keeps order within one queue, but retries can break it. So consumers shouldn't depend on order — for example, we check the current state in the database instead of trusting that events arrive in sequence."
- **"Why RabbitMQ and not Kafka?"** → "Our use case was **task-style messaging** — do this job, send this notification — with routing, retries and dead-lettering. RabbitMQ is great at that and simple to run. Kafka is better for a **high-volume event log** you replay or stream-process. We didn't need that." (CV says "Kafka basics" — *don't claim more.*)

Drill 5 — "What would you do differently?"

Why they ask: To see **self-awareness**. "Nothing" is the worst answer.

Strong answer:

"Three things. **One — distributed tracing from day one.** When a request crosses the API, RabbitMQ and a service, a single log file isn't enough. I'd add **OpenTelemetry** with a **correlation ID** passed in every request and every message, so we can follow one user action across all services. We had to add correlation IDs later, and debugging was painful before that. [FILL: only if true — otherwise 'we still rely mostly on logs, and this is what I'd add'] **Two — the outbox pattern from the start,** so an event is never lost between the database and the broker. **Three — contract tests between services,** so if the core changes an event's shape, the consumer's test fails in CI, not in production. Overall, the architecture was right, but I'd invest in **observability and contracts earlier,** because in a distributed system they're not optional."

Bonus drills (they might ask)

Question	Short strong answer
"How do the React apps authenticate with the API?"	"Token-based auth with Laravel Sanctum. The tenant app and admin app hit different route groups with different middleware — admin routes go through the landlord context, tenant routes through tenant middleware."
"Would you choose microservices again?"	"For reports and notifications, yes — they really needed independent scaling. For a new small product, I'd start with a modular monolith and split only when there's a clear reason."
"How big was the team, how long did it take?"	"[FILL: X engineers, Y months]. I led the architecture and [FILL: which parts you coded yourself]."

Trade-off line

- **Main:** *"Microservices gave us independent scaling and independent deploys, but the cost is operational complexity: more things to monitor, network failures between services, and eventual consistency."*
- **Alt 1:** *"We traded simplicity for isolation. A slow report can't hurt order taking anymore, but debugging now crosses service boundaries."*
- **Alt 2:** *"The data in the report service is eventually consistent — usually seconds behind. For reports that's fine; for payments it wouldn't be, so payments stayed in the core."*

Numbers to know

Number	Source	How to defend	Safe phrasing
~20% server load reduction	CV	Drill 3 — metric + period	"about 20% lower load on the core servers"
100+ clients	CV	Count of tenant DBs in the landlord DB	✓ safe
3 services + core + 2 React apps	CV	Report, Notification, SOP	✓ safe
Zero downtime	CV	"Both systems in parallel, step-by-step traffic move"	✓ if true
Team size	[FILL]	—	"a small team of [X]"

Number	Source	How to defend	Safe phrasing
Duration	[FILL]	—	"about [X] months"

Key phrases

Phrase	When to use	Example
"we ran both systems in parallel"	Explaining low-risk migration	"We ran both systems in parallel, so we could roll back any endpoint."
"a zero-downtime cutover"	The result	"The final cutover was zero-downtime — clients didn't notice."
"service boundaries"	Drill 1	"We drew the service boundaries around load and data ownership."
"eventual consistency"	Trade-off	"Reports are eventually consistent, usually a few seconds behind."
"at-least-once delivery"	Drill 4	"RabbitMQ is at-least-once, so our consumers are idempotent."
"the strangler pattern"	If they ask the migration style	"It was basically the strangler pattern — replace the old system piece by piece."

C2 ● Reports up to 85% Faster (S2)

Opening: "Tell me about a performance problem you solved."

Model answer (STAR, ~2 min):

S: "Our heaviest reports ran over millions of records — [FILL: which report, e.g. the sales or order summary]. It took about [FILL: X] seconds, and for large clients it sometimes **timed out**. **T:** I owned making it fast and reliable. **A:** First I **measured instead of guessing**: I ran EXPLAIN on the slowest queries and counted queries per request. I found [FILL: full table scans / N+1 queries / both]. I added **composite indexes**, fixed the **N+1 queries** with eager loading, and moved the heavy aggregation reads to **MongoDB aggregation pipelines**. Large reports became **queued jobs** with retries, a dead-letter queue and priorities, and results were **cached in Redis**. **R:** Load time dropped by **up to 85%** — from about [FILL: X]s to [FILL: Y]s on the heaviest reports — and we keep **99.9%+ availability** for report generation."

Drill 1 — "What did EXPLAIN show you?"

Why they ask: Anyone can say "I used EXPLAIN". They want to hear that you can **read** it.

Strong answer (fill in the real case; the columns below are how EXPLAIN works in MySQL):

"The main problem was a **full table scan**. EXPLAIN showed **type: ALL**, **key: NULL** — so no index was used — and the **rows** column estimated around [FILL: number] rows examined for a query that returned only a few hundred. The **Extra** column also showed '**Using temporary; Using filesort**',

which means MySQL was building a temporary table and sorting it on disk for the GROUP BY and ORDER BY. After adding the composite index, EXPLAIN showed **type: range** or **ref**, the new index in the **key** column, and **rows** dropped to [FILL: number]. The filesort went away because the index already had the right order."

Quick EXPLAIN cheat-sheet (know these cold):

Column	Bad sign	Good sign
type	ALL (full scan), index (full index scan)	const , eq_ref , ref , range
key	NULL	your index name
rows	close to table size	small
Extra	Using filesort , Using temporary	Using index (covering index)

If they push further:

- **"EXPLAIN vs EXPLAIN ANALYZE?"** → "EXPLAIN shows the **plan and estimates**. EXPLAIN ANALYZE (MySQL 8.0.18+) actually **runs** the query and shows real times and row counts — useful when estimates are wrong."

Drill 2 — "Which composite index, and why that column order?"

Why they ask: Column order is the most common index mistake. This checks real understanding.

Strong answer:

"For example, a query like 'orders for this distributor, in this date range, grouped by status'. [FILL: your real table and columns.] The index was on **(distributor_id, created_at)** — the **equality column first, then the range column**. The reason is the **leftmost-prefix rule**: MySQL can use a composite index from the left. With **distributor_id** first, it jumps straight to that distributor's rows, and inside that part the rows are already **sorted by date**, so the date range is one continuous scan. If I put **created_at** first, MySQL would scan every distributor's rows in that date range and filter afterwards — much more work. Where possible I made it a **covering index** — adding the selected columns so MySQL answers from the index alone, without reading the table. That shows as 'Using index' in EXPLAIN."

If they push further:

- **"Downside of more indexes?"** → "Every index **slows down writes** — each INSERT and UPDATE must update it — and uses disk and memory. So I only add indexes for queries that really matter, and check for unused ones."
- **"Do you need tenant_id in the index?"** → "Not in our case — each tenant has its **own database**, so the data is already separated. In a shared-table design, **tenant_id** would almost always be the first column."

Drill 3 — "How do you invalidate the cache?"

Why they ask: "There are only two hard things in computer science: cache invalidation and naming things." They want to see you've thought about **stale data**.

✂ Pick your real version:

Option A — TTL-based (simplest, most common for reports):

"Mainly **TTL-based**. The cache key contains the **tenant**, the **report type** and a **hash of the filters** — date range, territory, and so on — so two users with different filters never get each other's results, and two tenants never share a key. Each report type has a TTL [FILL: e.g. 5–30 minutes] that we agreed with the product team. For reports, a few minutes of staleness is acceptable, and TTL is simple and hard to get wrong."

Option B — Event-based + TTL:

"TTL as a safety net, plus **event-based invalidation**. When data that feeds a report changes — for example, an order is approved — we clear the related keys [FILL: by a key prefix / cache tags]. The TTL makes sure that even if we miss an event, the data is never stale for long."

If they push further:

- **"What's a cache stampede and how do you prevent it?"** → "When a popular key expires, many requests hit the database at the same time to rebuild it. Fixes: a **lock** so only one request rebuilds it while others wait or get the old value, or **random jitter on TTLs** so keys don't all expire together." [FILL: which one you used, or 'we'd add a lock']
- **"How do you show the user it's cached?"** → "A 'generated at' time on the report, and a refresh button for users who need live data." [FILL: only if true]

Drill 4 — "Why MongoDB and not just better MySQL?"

Why they ask: Adding a second database is a big decision. They want the **reason**, not "it's faster".

Strong answer:

"We did optimize MySQL first — indexes and N+1 fixes gave a big part of the win. But some reports were **aggregation-heavy**: grouping millions of rows by territory, product and date, with many joins. Those queries were **read-only**, the data could be **denormalized**, and they were competing with live order writes on the same MySQL server. So we moved **only those heavy reads** to MongoDB, with documents already shaped for the report, and used **aggregation pipelines** — **\$match**, **\$group**, **\$project** — to compute them. **MySQL stayed the source of truth** for transactions. So it wasn't 'MongoDB is better', it was 'separate the read workload from the write workload'."

If they push further:

- **"How does MongoDB stay in sync with MySQL?"** → ✂ [FILL: A — "a scheduled job copies/aggregates new data every X minutes" · B — "events from the core over RabbitMQ update MongoDB" · C — "the report service writes both"]. Then: "The trade-off is a small delay — reports can be a few minutes behind."
- **"Why not a read replica?"** → "A replica takes load off the primary, but the query is still expensive — it would just be slow somewhere else. The document model made the aggregation itself cheaper." [FILL: if you also used replicas, say so]

Bonus drills

Question	Short strong answer
"How did you find the N+1 queries?"	"Laravel Telescope shows every query per request. A report page running 500 queries instead of 5 is an obvious N+1. Fix: eager loading with <code>with()</code> / <code>load()</code> ."
"Why queue the report instead of running it in the request?"	"A 60-second HTTP request will time out and holds a worker. As a queued job, the user gets 'your report is being prepared', the job retries on failure, and big reports can't block small ones — we used priority queues for that."
"What's in the dead-letter queue, and who looks at it?"	"Jobs that failed after all retries. We alert on it, check the error, fix the cause, and re-run them." [FILL: who owns it]
"How is 99.9% availability measured?"	"Successful report generations divided by total requests, over [FILL: 30 days]. 99.9% means roughly 1 failure in 1,000."
"Pagination on millions of rows?"	"Avoid big OFFSETs — MySQL still reads and throws away all the skipped rows. Use keyset pagination : <code>WHERE id > last_seen_id ORDER BY id LIMIT 50</code> ."

Trade-off line

- **Main:** "Caching makes reports fast, but the trade-off is freshness. Users might see data a few minutes old, so we agreed the TTL with the product team."
- **Alt 1:** "Moving reads to MongoDB made reports fast, but now we have two databases to keep in sync — and that sync is the new thing that can break."
- **Alt 2:** "Indexes speed up reads but slow down writes, so I only added them for the queries that really mattered."

Numbers to know

Number	Source	How to defend	Safe phrasing
Up to 85% faster	CV	Before → after on the heaviest report: [FILL: X s → Y s]	"up to 85% on our heaviest reports"
99.9%+ availability	CV	Success rate of report jobs over [FILL: period]	"we held our 99.9% SLO"
"Millions of records"	CV	[FILL: roughly how many rows in the biggest table?]	"several million rows for our biggest tenants"
Query count before/after	[FILL]	Telescope	"hundreds of queries down to a handful"

⚠ **"Up to" matters.** Always say "up to 85%" — it's the best case, not the average. If asked about the average: "Most reports improved by [FILL]; 85% was the heaviest one."

Key phrases

Phrase	When to use	Example
"I measured before optimizing"	Start of Action	"I measured before optimizing — EXPLAIN first, then changes."
"a full table scan"	Drill 1	"EXPLAIN showed a full table scan on the orders table."
"the leftmost-prefix rule"	Drill 2	"Because of the leftmost-prefix rule, the equality column goes first."
"a covering index"	Drill 2	"With a covering index, MySQL never touches the table."
"the source of truth"	Drill 4	"MySQL stayed the source of truth; MongoDB was a read model."
"dead-letter queue"	Queues	"Failed jobs end up in a dead-letter queue we can replay."

C3 On-call, SLIs/SLOs, 65% Faster Resolution (S4)

Opening: "Tell me about a production incident you handled."

Model answer (STAR, ~2 min):

S: "I'm the primary 24/7 on-call engineer for six production apps. [FILL: one real incident — what broke, when, who was affected. e.g. 'One evening, report generation started failing for several large clients.'] **T:** I had to restore service fast, keep clients informed, and make sure it didn't happen again. **A:** [FILL: how you detected it — which alert or dashboard.] I checked [FILL: Grafana / CloudWatch / Telescope / logs] and found the root cause: [FILL]. The immediate fix was [FILL: rollback / restart / config fix / hotfix]. While I worked on it, I [FILL: posted updates in Slack for the team and the account managers]. Afterwards I [FILL: added an alert / a test / a runbook] so it can't happen the same way again. **R:** Service was back in [FILL: time]. More broadly, when we moved from raw CPU alerts to **SLO-based alerts** — 99.9% report availability, API latency and error rate — our **issue resolution time dropped by 65%.**"

 If you don't have one clean incident story, use a **common incident pattern** you really saw (a queue backlog, a slow query after a data spike, a full disk, a failed deploy) and tell it honestly.

Drill 1 — "What's the difference between an SLI, an SLO and an SLA?"

Why they ask: It's a definition test. Answer **fast and clean**, with an example.

Strong answer:

"An **SLI** — service level *indicator* — is a **measurement**. For example: the percentage of report requests that succeed, or the p95 API latency. An **SLO** — *objective* — is our **internal target** for that measurement. For example: 99.9% of report generations succeed over 30 days, or p95 latency under [FILL] milliseconds. An **SLA** — *agreement* — is a **promise to the customer**, usually with a penalty like service credits if we miss it. The SLA is always looser than the SLO, so we get warned internally before we break a promise. The SLO also gives you an **error budget**: 99.9% over 30 days means about **43 minutes** of failure allowed. If we're burning the budget fast, we slow down new features and focus on reliability."

Math to know: $99.9\% / 30 \text{ days} \approx 43 \text{ min} \cdot 99.99\% \approx 4.3 \text{ min} \cdot 99\% \approx 7.2 \text{ hours}$.

Drill 2 — "How was the 65% measured?"

 **Pick your real version:**

Option A — MTTR from a ticket or incident tracker:

"We measured **mean time to resolve** — from the first alert or client report until the fix was confirmed — using [FILL: Jira / our incident log]. We compared [FILL: the X months before] SLO alerting with [FILL: the X months after]. Average resolution went from about [FILL] to about [FILL], which is roughly 65% faster."

Option B — Honest estimate:

"It was calculated from our issue tracker — the time between an issue opening and closing — before and after we introduced SLO-based alerts and dashboards. It's not a perfect metric, because incidents differ, but the trend over [FILL: months] was consistent."

Then explain WHY it got faster (this is the valuable part):

"Three reasons. **We found problems earlier** — alerts fired on user-facing symptoms, often before clients called. **We found the cause faster** — structured logs with request IDs, plus Telescope, let us go from 'error rate is up' to the exact failing query in minutes. And **runbooks** — for common problems, the fix was written down, so anyone on the team could apply it."

Drill 3 — "How do you avoid alert fatigue?"

Why they ask: Too many alerts means people ignore them — and miss the real one.

Strong answer:

"The main rule: **alert on symptoms, not causes**. A CPU spike at 2 AM that users don't feel shouldn't wake anyone up. An error rate above the SLO, or report generation failing, should. Practically: first, **every alert must be actionable** — if nobody needs to do anything, it's a dashboard, not an alert. Second, **severity levels** — pages for user impact, Slack messages for warnings, weekly review for trends. Third, we **review noisy alerts** and delete or tune the ones that fire without a real problem. [FILL: e.g. 'We cut the number of alerts from X to Y' — only if true.]"

If they push further:

- **"What's burn-rate alerting?"** → "Instead of alerting on one bad minute, you alert when you're using the error budget **too fast** — for example, if at this rate you'd use a month's budget in two days. It's much less noisy."

Bonus drills

Question	Short strong answer
"Walk me through debugging a slow API at 3 AM."	"Check the dashboard — is it one endpoint or all? One tenant or all? Did something change — a deploy, a data spike? Then APM or Telescope for slow

Question	Short strong answer
	queries, then logs by request ID. Mitigate first — roll back or scale — then find the root cause."
"What's in a postmortem?"	"Timeline, impact, root cause, what went well, what went badly, and action items with owners. Blameless — we fix the system, not the person."
"How do you balance on-call with feature work?"	"Runbooks and good alerts keep on-call quiet. When it's noisy, that's a signal to spend time on reliability, not just on features."
"Six apps — which ones?"	[FILL: e.g. core API, tenant app, admin app, report service, notification service, benchmark API]

Trade-off line

- **Main:** *"A 99.99% target sounds nice, but every extra nine costs a lot of engineering. 99.9% was the right balance for reports."*
- **Alt 1:** *"Fewer alerts means we might notice some small issues later — but the alerts we do have, people actually trust and act on."*
- **Alt 2:** *"Mitigate first, investigate second. A rollback isn't elegant, but it gets clients working while we find the real cause."*

Numbers to know

Number	Source	How to defend	Safe phrasing
65% faster resolution	CV	Drill 2: MTTR before/after, period	"resolution time dropped by about 65%"
6 production apps	CV	Name them (bonus drill)	✓ safe
99.9% report availability SLO	CV	≈ 43 min error budget / 30 days	✓ safe
Incident: time to restore	[FILL]	Your incident story	"back within about [X]"

Key phrases

Phrase	When to use	Example
"root cause"	Incident story	"The root cause was a missing index after a data spike."
"a blameless postmortem"	After the incident	"We wrote a blameless postmortem and added two action items."
"error budget"	SLO talk	"When the error budget runs low, reliability work comes first."
"we alert on symptoms, not causes"	Alert fatigue	"We alert on symptoms, not causes — users don't feel CPU."

Phrase	When to use	Example
"mitigate first"	Incident process	"I always mitigate first — roll back — then investigate."
"time to resolve (MTTR)"	65% number	"Our mean time to resolve dropped by about 65%."

C4 Vue → React/Next/RTK/TypeScript Rewrite: Leadership (S6)

Opening: *"Tell me about a time you led a team or set technical standards."*

Model answer (STAR, ~2 min):

S: "Our frontend was on Vue.js, and [FILL: why rewrite? e.g. the Vue 2 code was hard to maintain / inconsistent patterns across pages / Vue 2 was reaching end of life / hard to hire for]. **T:** I led the rewrite to **React, Next.js, Redux Toolkit and TypeScript**, and we had **two months** to deliver. **A:** First, I **cut the scope**: we focused on the key user journeys, not every screen. I worked with **UX designers and product** to rebuild those journeys from Figma. Then I set the **standards** — component structure, folder layout and code-review rules — and **documented them**, so the team could follow them without asking me every time. [FILL: team size, and how you split the work — e.g. by module or by journey.] **R:** We shipped in two months, the team adopted the standards, and **user engagement went up 30%.**"

Drill 1 — "How did you fit a rewrite into two months?"

Why they ask: Rewrites are famous for going over time. They want to hear **how you managed risk**.

Strong answer:

"Three things made it possible. **Scope.** We didn't rewrite everything. We listed all screens, looked at [FILL: usage data / feedback from product], and picked the **journeys users use most** — [FILL: e.g. order taking, dashboards, reports]. Less-used screens came later. **Foundation first.** In the first [FILL: week or two], I set up the project skeleton — routing, API layer, Redux store, shared components, lint rules. After that, every developer built pages on the same base, so work could run **in parallel** without conflicts. **Small releases.** [FILL: A — 'We released journey by journey, so users got new pages as they were ready' / B — 'We released it all together, but tested each journey with product as soon as it was done.'] The backend API already existed, so this was a **frontend-only** change — that also kept the risk down."

Drill 2 — "How did you measure the 30% engagement increase?"

Why they ask: "Engagement" is vague. They want the **exact metric**.

 **Pick your real version:**

Option A — Active users:

"We tracked [FILL: daily / weekly] active users in [FILL: Google Analytics / our own event logs]. Comparing [FILL: X weeks] before and after launch, active users rose about 30%."

Option B — Actions per session:

"We measured key actions per user — for example [FILL: orders created, reports opened] — from our own backend data. After the new UI, those went up about 30%."

Option C — Honest:

"It came from the product team's analytics — I didn't set up the measurement myself. The number they reported was about 30% higher engagement after launch."

If they push further:

- "How do you know it was the rewrite, not something else?" → "Fair point — we can't fully isolate it. Nothing else big launched at the same time [FILL: confirm], and the increase was on the pages we rebuilt. But I'd say the rewrite **contributed to** it, not that it was the only cause."

Drill 3 — "What if someone on the team didn't agree with your standards?"

Why they ask: Leadership = influence without forcing. They want to see **openness**.

Strong answer:

"I'd listen first. Standards are for the team, not for my ego. If someone has a better idea, we change the standard — and that happened: [FILL: a real example, e.g. 'a teammate suggested a better folder structure for feature modules, and we adopted it']. If it's just a personal preference, I explain **why** the standard exists — usually consistency, so anyone can open any page and understand it quickly. I also made standards **cheap to follow**: ESLint and Prettier rules run automatically, a PR template with a checklist, and example components to copy. When the tools enforce the style, code review can focus on logic, not on formatting arguments."

Drill 4 — "Why Redux Toolkit and not just Context?"

Why they ask: To check you chose tools for **reasons**.

Strong answer:

"Context is great for **small, rarely-changing** values like the current user or theme. But when a Context value changes, **every consumer re-renders**, and it has no built-in tools for loading states, caching or debugging. Our app had a lot of **shared server state** across many pages — tenant settings, filters, lists of orders and products. Redux Toolkit gave us **predictable updates** through slices, **DevTools** to see every state change, and far less boilerplate than old Redux. [FILL: If you used RTK Query: 'RTK Query also handled API caching and loading states for us.'] Today I'd also consider **React Query** for pure server state — it handles caching and refetching very well — and keep Redux for real client state."

Bonus drills

Question	Short strong answer
"Why Next.js?"	[FILL: SSR/SEO for public pages? file-based routing? team preference?] — "...it gave us routing and build tooling out of the box, so we didn't build that ourselves."

Question	Short strong answer
"Why TypeScript?"	"Types catch mistakes at build time — like a wrong API field — instead of in production. On a team, types also act as documentation for component props and API responses."
"How did you work with UX and product?"	"We reviewed Figma designs together before coding, agreed on the journey, and I raised technical constraints early — for example, when a design needed data the API didn't have yet."
"What was the hardest part?"	[FILL: e.g. keeping old and new in sync / state design / getting the team used to TypeScript]
"What did you do yourself vs. the team?"	[FILL: e.g. 'I built the foundation and the hardest journey, and reviewed every PR.']

Trade-off line

- **Main:** *"A rewrite is risky because you stop shipping features for a while. We reduced that risk by keeping the scope small and releasing journey by journey."*
- **Alt 1:** *"Strict standards slow people down in the first week, but after that everyone moves faster, because every page looks the same."*
- **Alt 2:** *"TypeScript adds some upfront effort, but it pays back the first time a type error catches a broken API field before it reaches users."*

Numbers to know

Number	Source	How to defend	Safe phrasing
2 months	CV	Scope cut + foundation first	✅ safe
+30% engagement	CV	Drill 2: which metric, which tool	"engagement rose around 30% after launch"
Team size	[FILL]	—	"a team of [X] frontend developers"
Journeys rebuilt	[FILL]	—	"the [X] most-used journeys"

Key phrases

Phrase	When to use	Example
"I cut the scope"	Two-month question	"I cut the scope to the journeys users touch every day."
"the team adopted the standards"	Result	"The team adopted the standards, and PR reviews got faster."
"we worked closely with UX and product"	Collaboration	"We worked closely with UX and product from Figma to release."

Phrase	When to use	Example
"cheap to follow"	Standards	"I made the standards cheap to follow — lint rules do the work."
"shared server state"	Redux question	"We had a lot of shared server state across pages."

C5 Multi-tenancy + Payments + POS (S3, S5, SoftArch)

This block has **three openings**. They may ask one, two or all three.

C5-A — Multi-tenancy

Opening: *"How does multi-tenancy work in your system?"*

Model answer (~90 s):

"Each client company gets its **own tenant database**, and a central **landlord database** holds platform data like organizations and billing. When a request comes in, middleware **resolves the tenant** — [FILL: from the domain / a header] — and switches the database connection to that tenant's database. On top of that, **every cache key and every queued job carries the tenant scope**, and we have **territory-scoped RBAC**, so users only see their own region's data. I built this from scratch, and across **100+ tenants** we've had **zero cross-tenant data leaks**."

Drill A1 — "How does a queued job know which tenant it belongs to?"

Strong answer:

"The request context is gone when the job runs — the job runs later, in a worker process. So the **tenant ID is part of the job payload**. When the job starts, it **switches the connection** to that tenant's database, does its work, and **always switches back** to the landlord connection at the end — in a **finally** block, so it happens even if the job fails. Otherwise the next job on the same worker could run against the wrong tenant, which is exactly the kind of leak we must avoid. Same idea for **cache keys**: the tenant ID is in every key, so two tenants can never read each other's cached reports."

Drill A2 — "What's the downside of database-per-tenant?"

Strong answer:

"Three main costs. **Migrations** — a schema change must run on 100+ databases. We run them with a command that loops over tenants, and every migration must be **backward-compatible**, because during the rollout some tenants are on the new schema and some on the old one. [FILL: how you handle a migration that fails for one tenant.] **Cross-tenant reporting** — you can't do one SQL query across all clients. For platform-wide analytics we loop over tenants or copy data to a separate analytics store. **Resources** — more databases means more connections and more to back up and monitor. For us, it was worth it: enterprise clients get **strong isolation**, and we can move a big tenant to its own server if needed."

Drill A3 — "Why not a shared DB with a tenant_id column?"

Strong answer:

"Shared tables are **cheaper and simpler** to operate — one schema, one migration, easy cross-tenant queries. But isolation depends on **every single query** having the right **WHERE tenant_id = ?**. One missed clause in one report, and one client sees another client's data. You can reduce that risk with global scopes, but it's still one bug away from a leak. With database-per-tenant, a wrong query can only see **its own tenant's data**, because the connection itself is scoped. For enterprise clients with sensitive sales data, that stronger isolation was worth the extra operational cost."

If they push further:

- **"How does territory RBAC work?"** → "On top of roles and permissions, each user is assigned territories. Queries for orders, customers and reports are **filtered by the user's territories** at the service layer, so a regional manager only sees their region — [FILL: done with a query scope / middleware]."
- **"How did you test for leaks?"** → [FILL: e.g. 'tests that create two tenants and assert tenant A can't read tenant B's data'] — if none: "Mostly code review and the structure itself. Today I'd add automated isolation tests for every new endpoint."

C5-B — Payments

Opening: *"How did you make payments reliable?"*

Model answer (~90 s):

"I designed the payment gateway integration for billing and subscriptions: **initiation, webhooks, retries, auto-reconciliation and receipts**. Two things make it safe. **Every webhook is verified with an HMAC signature**, so nobody can fake a 'payment succeeded' call. And **every request carries an idempotency key**, so if the gateway retries or sends a duplicate callback, we never charge twice or activate twice. A **reconciliation job** catches anything that drifts. We've had **zero failed transactions** in production."

Drill B1 — "How does HMAC verification work, exactly?"

Strong answer:

"The gateway and our server share a **secret key**. When the gateway sends a webhook, it computes an **HMAC-SHA256** of the request body using that secret, and puts the result in a header. On our side, we compute the same HMAC over the **raw request body** and compare the two with a **constant-time comparison** — **hash_equals** in PHP — so an attacker can't guess the signature byte by byte from timing. If they don't match, we reject with 401 and log it. Without the secret, nobody can create a valid signature — so a fake 'payment succeeded' request is rejected. [FILL: If the gateway also sends a timestamp: 'We also reject old timestamps, to block replay attacks.']"

Drill B2 — "What if the same webhook arrives twice?" / "How does the idempotency key work?"

Strong answer:

"Duplicates are normal — gateways retry when they don't get a fast 200. So we handle it on two levels. **Outgoing requests:** when we start a payment, we send an **idempotency key**, unique per

payment attempt. If our request times out and we retry, the gateway sees the same key and returns the original result instead of charging again. **Incoming webhooks:** we store the gateway's transaction ID with a **unique constraint**. If the same webhook comes again, the insert fails or we find the existing record, and we just return 200 without doing anything again. We also treat payment status as a **state machine** — pending → paid, or pending → failed. A payment that's already 'paid' can't go back, so an old or out-of-order webhook can't break the state."

Drill B3 — "What if the webhook never arrives?"

Strong answer:

"That's why we have **reconciliation**. A scheduled job looks for payments that are still 'pending' after [FILL: X minutes], **asks the gateway's API** for their real status, and updates our records. It also compares our records with the gateway's [FILL: daily settlement report], so any mismatch is flagged. So the webhook is the fast path, and reconciliation is the safety net."

Drill B4 — "Which gateway?" / "What does 'zero failed transactions' mean?"

"[FILL: gateway name]." And: "It means no payment was lost or double-charged because of our system over [FILL: period]. Payments declined by the bank — like insufficient funds — are normal and not counted as failures of our system."

⚠ Be ready for this. "Zero failed" sounds too perfect — the clarification above makes it believable.

C5-C — GirlyShopper POS (SoftArch)

Opening: *"Tell me about GirlyShopper."*

Model answer (~60–90 s):

"At SoftArch I built **GirlyShopper**, a multi-store e-commerce platform with **POS integration**, barcode inventory and real-time stock tracking, using Laravel, Vue and MySQL. Before, the shop staff reconciled online and in-store stock **by hand**. With one shared stock system for both channels, manual stock reconciliation dropped by **about 60%**. I also integrated **card, bKash, Nagad and cash-on-delivery** payments, with webhook-driven order status updates."

Drill C1 — "How did you keep stock correct between online orders and the POS?"

Why they ask: This is the classic **race condition** question — two sales of the last item at the same time.

✂ **Pick your real version** (both are correct techniques):

Option A — Atomic conditional update:

"Stock is decreased with a **single atomic UPDATE**: `UPDATE products SET stock = stock - :qty WHERE id = :id AND stock >= :qty`. If two sales try to take the last item at the same moment, MySQL runs the updates one after the other. The first one changes one row, and the second one changes **zero rows** — so we know it's out of stock and reject that sale. There's no read-then-write gap, so we can't oversell."

Option B — Row lock in a transaction:

"Inside a database transaction, we read the product with **SELECT ... FOR UPDATE**, which locks that row. We check the stock, decrease it, and commit. A second sale for the same product waits for the lock, then sees the updated stock. It's a bit slower than the atomic update, but useful when the check has more logic."

If they push further:

- **"What if the POS is offline?"** → [FILL: e.g. 'The POS required a connection' / 'It queued sales locally and synced later; conflicts were flagged for staff.']
- **"How was the 60% measured?"** → [FILL: e.g. hours per week spent on manual reconciliation, before vs. after]

Trade-off lines (C5)

- **Multi-tenancy:** *"Database-per-tenant costs more to operate — migrations and connections — but a missing WHERE clause can never leak one client's data to another."*
- **Payments:** *"Webhooks are fast but unreliable, so they're never the only path — reconciliation is slower, but it's always right in the end."*
- **POS:** *"Locking protects correctness, but it adds waiting under heavy load. For stock, correctness wins — overselling is worse than a few milliseconds."*

Numbers to know

Number	Source	How to defend	Safe phrasing
100+ tenants, zero leaks	CV	Tenant count in landlord DB	✅ safe
40+ industries	CV	Client list	✅ safe
Zero failed transactions	CV	Drill B4 definition + [FILL: period]	"no payment lost or double-charged in production"
~60% less manual reconciliation	CV	[FILL: hours per week before/after]	"much less manual reconciliation, about 60%"
4 payment methods (card, bKash, Nagad, COD)	CV	—	✅ safe

Key phrases

Phrase	When to use	Example
"strong isolation"	Multi-tenancy	"Database-per-tenant gives us strong isolation."
"tenant context"	Jobs, caching	"Every job carries its tenant context in the payload."
"HMAC signature verification"	Webhooks	"Every webhook goes through HMAC signature verification."
"idempotency key"	Duplicates	"An idempotency key means a retry never charges twice."

Phrase	When to use	Example
"reconciliation is the safety net"	Missing webhook	"Webhooks are the fast path; reconciliation is the safety net."
"a race condition" / "oversell"	POS stock	"An atomic update prevents a race condition, so we never oversell."

C6 REST/SOAP APIs + Client Integrations (S7)

Opening: *"Tell me about working with a difficult client integration."*

Model answer (STAR, ~2 min):

S: "I designed the REST APIs behind our mobile field-force app, plus **SOAP endpoints and queue-based sync** for legacy enterprise clients. [FILL: one specific client problem — e.g. 'One large client's ERP kept sending orders that failed to sync, and their team thought our API was down.'] **T:** I needed to find the real cause quickly and keep the client's trust. **A:** First I **reproduced it:** I took their actual payload from our logs, using the [FILL: request ID], and replayed it in Postman. I found [FILL: the cause — e.g. a date format / a missing field / a timeout on large batches]. Then I **communicated clearly:** a short call with their technical team to agree on the fix, and a **written summary** afterwards — what happened, why, what we changed, what they needed to change. **R:** [FILL: it was fixed within X, and what you changed so it wouldn't happen again — e.g. better validation errors, a retry queue, a health check.]"

Drill 1 — "How do you explain a technical problem to a non-technical client?"

Strong answer:

"I use a simple order: **impact, cause, action, time**. **Impact** first — 'Yesterday, about 200 orders from your system didn't reach ours.' **Cause** in one plain sentence — 'Your system started sending dates in a different format, and our system rejected them.' **Action** — 'We've fixed our side to accept both formats, and resent all 200 orders.' **Time** — 'Everything is back to normal as of this morning, and we've added an alert so we'll know immediately if it happens again.' No jargon, no blame, and I always end with what **prevents** it next time. Then I send the same thing **in writing**, because with international clients in different time zones, a written summary is what people actually read."

Drill 2 — "Why SOAP? Isn't that old?"

Strong answer:

"Yes, it's old — and that's exactly why. Some **enterprise clients run legacy ERP systems** that only speak SOAP. We don't choose their technology; we meet them where they are. So we exposed SOAP endpoints for them, but **behind** those endpoints it's the same service logic as our REST API. And we put a **queue** in between, so if their system sends a big batch or our side is slow, nothing is lost — messages are processed and retried in the background."

If they push further:

- **"REST vs SOAP?"** → "SOAP is a strict XML protocol with a formal contract (WSDL) and built-in standards — good for enterprise tools. REST is lighter: JSON over HTTP, using HTTP methods and status codes. For new APIs I always choose REST."

Drill 3 — "How do you design an API for a mobile app with bad connectivity?"

Very relevant for Field Nation (technicians in the field).

Strong answer:

"Field staff often have weak signal, so I design for that. **Small payloads** — only the fields the screen needs, paginated lists. **Idempotent writes** — the app sends a client-generated ID with each order or check-in, so if a request is retried after a timeout, the server doesn't create a duplicate. **Offline-friendly sync** — [FILL: if true: 'the app stores actions locally and syncs when the connection comes back, and the API accepts batches']. **Versioning** — mobile users don't update immediately, so old app versions keep working. We add fields instead of changing them, and version the API when we have to break something."

Bonus drills

Question	Short strong answer
"How do you version an API?"	"URL versioning like <code>/api/v1/</code> is simple and clear. Within a version, only additive changes — new optional fields, never removing or renaming."
"What goes in your tech specs?" (CV)	"Data model, API contract — endpoints, request and response shapes, errors — UI flow, and a test plan. Written before coding, so product, frontend and mobile agree first."
"How do you work with offshore teams?" (CV)	"Async and written: clear Jira tickets, detailed PR descriptions, docs, and Slack updates at the end of my day so they can continue in theirs."
"HTTP status codes you use?"	"200/201 success, 400 bad input, 401 not logged in, 403 no permission, 404 not found, 409 conflict, 422 validation error, 429 rate limit, 500 server error."

Trade-off line

- **Main:** *"Supporting SOAP costs us extra maintenance, but refusing it would mean losing enterprise clients. We kept the cost down by sharing one service layer."*
- **Alt:** *"A queue in front of client integrations adds a small delay, but nothing gets lost when their system sends a burst."*

Numbers to know

Number	Source	How to defend	Safe phrasing
Client problem timeline	[FILL]	Your story	"fixed within [X] days"
Clients on SOAP	[FILL]	—	"a few legacy enterprise clients"

Key phrases

Phrase	When to use	Example
"I reproduced it first"	Debugging	"I reproduced it first with their real payload."
"impact, cause, action, time"	Non-technical explanation	"I always explain it as impact, cause, action, time."
"meet them where they are"	SOAP	"We meet enterprise clients where they are — sometimes that's SOAP."
"a written summary"	Communication	"After every call, I send a written summary."
"backward-compatible"	Versioning	"Every change to the mobile API is backward-compatible."

C7 🟡 Field-force Modules + GPS-Spoofing Detection (S9) — your "Why Field Nation" story

Opening: *"What feature are you most proud of?"*

Model answer (STAR, ~2 min):

S: "Sokrio's clients have large field teams — sales reps and van drivers visiting shops every day. Managers had no reliable way to plan their work or know if they really visited the shops. **T:** I built the field-force side of the platform. **A: Task Manager** for planning, approval and achievement tracking; **Van Sales, Vehicle Feedback** and **Van Attendance** — all syncing in real time with the core platform. The most interesting part was **GPS-spoofing detection**, because some field staff used fake-location apps to mark attendance without being there. We detected it by [FILL: your real signals — see Drill 1]. **R:** Managers could finally trust attendance and visit data, and these modules **contributed to the company's 27% revenue growth**. And honestly, this is why Field Nation excites me: **technician check-in, location proof and work-order approval** are the same kind of problem at a bigger scale."

💡 The S line ("managers had no reliable way...") is a reasonable framing — adjust it if the real situation was different.

Drill 1 — "How did you detect GPS spoofing?"

Why they ask: It's an unusual and interesting claim — they'll want the details.

Strong answer — structure (say only the signals you really used):

"The key idea: **never trust the client alone**. The phone can lie, so we combine several signals and check them on the server."

[Pick the true ones:]

- **Mock-location flag** — Android marks locations that come from mock-location apps (`isFromMockProvider` / `isMock`), and the app sent that flag to the server.
- **Impossible movement** — we calculate the distance between consecutive points with the **haversine formula** and divide by time. If someone "moves" 50 km in 2 minutes, it's fake.

- **Geofence check** — check-in is only accepted within [FILL: X meters] of the shop's saved location.
- **Suspicious accuracy** — [FILL: e.g. perfectly identical coordinates every time, which real GPS never gives].
- **Device checks** — [FILL: rooted device / known spoofing apps installed].

"We didn't just block people automatically — [FILL: A: 'suspicious check-ins were flagged for the manager to review' / B: 'the check-in was rejected with a message']. That avoided punishing people for real GPS errors, like a weak signal indoors."

If they push further:

- **"What about false positives?"** → "GPS drifts indoors and in dense areas, so we used a tolerance, and [FILL: flagged for review instead of auto-rejecting]. A real person with a weak signal shouldn't lose a day's attendance."
- **"Can a smart user beat it?"** → "Yes, no client-side check is perfect — a rooted phone can hide the mock flag. That's why the **server-side** checks, like impossible speed and patterns over time, matter most. The goal is to make cheating hard and visible, not impossible."

Drill 2 — "What was *your* part of the 27% revenue growth?"

Why they ask: A company-level number on a personal CV — they want to see if you're honest.

Strong answer:

"I want to be careful here — **I didn't cause 27% revenue growth by myself**. That's company growth, and sales, product and the whole team drove it. My part was building the field-force modules — [FILL: e.g. 'Van Sales and Task Manager'] — which were [FILL: e.g. 'a key selling point for new enterprise clients / a paid add-on / the reason X clients signed']. So I'd say my modules **contributed to** that growth."

✓ This honest answer makes you **more** credible, not less. US interviewers respect it.

Drill 3 — "How did 'real-time sync' work?"

✂ **Pick your real version:**

- **A — Polling:** "The mobile app synced every [FILL] seconds/minutes and on important actions. Simple and battery-friendly."
- **B — Push:** "The server pushed updates through [FILL: WebSockets / FCM push notifications], and the app fetched the new data."
- **C — Event-based:** "Actions went to the API, which published events, so the core platform and dashboards updated within seconds."

Then: "For field users with weak signal, [FILL: the app queued actions offline and sent them when the connection came back]."

Why-Field-Nation bridge (memorize this)

"At Sokrio I solved **field workforce** problems: planning tasks, proving someone was really on site, approving the work, and syncing it all from a mobile app with weak connectivity. Field Nation solves

this for **technicians and work orders** at a much larger scale — check-in, location proof, approvals and payments. I'd love to bring that experience to a bigger marketplace."

Trade-off line

- **Main:** *"Stricter spoofing checks catch more cheating, but they also punish honest users with bad GPS. So we [FILL: flagged for review instead of auto-rejecting]."*
- **Alt:** *"Real-time sync gives managers fresh data, but costs battery and mobile data — so we synced on important actions, not constantly."* [only if true]

Numbers to know

Number	Source	How to defend	Safe phrasing
27% revenue growth	CV	Drill 2 — "contributed to"	" contributed to 27% company revenue growth"
Geofence radius	[FILL]	—	"within about [X] meters of the shop"
Number of field users	[FILL]	—	only say a number you know

Key phrases

Phrase	When to use	Example
"never trust the client"	Spoofing	"We never trust the client alone — the server checks too."
"impossible travel"	Spoofing	"Impossible travel between two points is a strong signal."
"flag for review"	False positives	"Suspicious check-ins are flagged for review, not auto-rejected."
"contributed to"	27%	"My modules contributed to 27% revenue growth."
"the same kind of problem at a bigger scale"	Why FN	"Field Nation is the same kind of problem at a bigger scale."

C8 Node.js Services

Opening: *"You list Node.js. What did you build with it?"*

Model answer (~60–90 s):

"Alongside the Laravel core API, I built **Node.js/Express services, background workers and developer tooling** in JavaScript and TypeScript. [FILL: one concrete example — e.g. 'a worker that consumes RabbitMQ events and sends notifications' / 'an internal CLI that sets up a new tenant' / 'a small Express service for ...']. I also built a **supply chain distribution system** as a project — Node.js, Express and React, managing products, customers and orders through REST APIs."

Drill 1 — "PHP vs. Node: when would you choose each?"

Strong answer:

"**Node** is strong for **I/O-heavy, concurrent** work — many open connections, real-time events, WebSockets, workers that wait on queues and APIs. Its **event loop** handles thousands of waiting connections in one process, because it doesn't block while waiting for I/O. **Laravel** is very productive for **CRUD-heavy business apps** — Eloquent, migrations, auth, queues and validation come built in. PHP's model is one request, one process, which is simple and safe: one bad request can't break the others. So at Sokrio: business logic and CRUD in Laravel, and Node where we needed long-running, event-driven processes. For Field Nation's move from PHP to Node, I already know both sides."

Drill 2 — "Explain the event loop."

Strong answer:

"Node runs your JavaScript on **one main thread**. When you do I/O — a database query, an HTTP call, reading a file — Node hands it off and **doesn't wait**. When the I/O finishes, its callback goes into a queue, and the event loop runs it when the main thread is free. The order: synchronous code first, then **microtasks** — `process.nextTick` and resolved promises — then **macrotasks** like timers and I/O callbacks. The big rule: **never block the main thread**. A heavy CPU loop — like a huge JSON transform or image processing — stops *every* request. For that you use **worker threads** or move the job to a queue."

Drill 3 — "How do you handle errors in async Node code?"

Strong answer:

"With `async/await`, I wrap risky code in `try/catch`. In Express, errors go to a **central error-handling middleware** — the one with four arguments, `(err, req, res, next)` — so every error returns the same JSON format and gets logged. In Express 4, errors in async handlers don't reach that middleware automatically, so you call `next(err)` or use a small wrapper; Express 5 handles rejected promises for you. I also handle **unhandled promise rejections** — log them and alert — because in modern Node they crash the process by default."

Bonus drills

Question	Short strong answer
"What is Express middleware?"	"A function <code>(req, res, next)</code> that runs in order for each request — auth, logging, validation, parsing. It either responds or calls <code>next()</code> ."
"How do you scale a Node service?"	"Run several processes — PM2 cluster mode or multiple containers behind a load balancer. Keep them stateless ; put sessions and cache in Redis."
"NestJS?"	"I'm learning it now. It's close to what I know — modules, controllers, services and dependency injection, similar to Laravel's structure." <i>(CV says learning — be honest.)</i>
"CommonJS vs ES modules?"	" <code>require / module.exports</code> vs <code>import / export</code> . ES modules are the standard now, with static analysis and top-level <code>await</code> ."

Trade-off line

- **Main:** "Node gives great concurrency for I/O, but one CPU-heavy task can block everything — so heavy work goes to worker threads or a queue."
- **Alt:** "Adding Node next to Laravel means two stacks to maintain, so we used it only where its strengths really mattered."

Numbers to know

Item	Source	Safe phrasing
Your concrete Node service	[FILL]	"for example, a worker that..."
Years of Node	CV lists Node in the 5+ years backend line	"[FILL] years of Node alongside Laravel" — don't overclaim

Key phrases

Phrase	When to use	Example
"I/O-bound vs CPU-bound"	PHP vs Node	"Node is great for I/O-bound work, not CPU-bound."
"don't block the event loop"	Event loop	"The first rule in Node: don't block the event loop."
"a central error handler"	Errors	"All errors go to a central error handler in Express."
"stateless services"	Scaling	"Stateless services are easy to scale horizontally."

C9 Docker + CI/CD (~40% faster deployments)

Opening: "Tell me about your deployment process."

Model answer (~90 s):

"I containerized our services with **Docker** and built **CI/CD pipelines** that run automated tests before every deploy. [FILL: CI tool — GitHub Actions / GitLab CI / Jenkins / Bitbucket Pipelines.] On every push, the pipeline runs [FILL: lint, tests, build the image], and on merge to the main branch it deploys [FILL: how — e.g. pulls the new image on the server and restarts the containers]. Deployments got **about 40% faster**, and we stopped having '**works on my machine**' problems, because every environment runs the same image. At SoftArch I also set up Dockerized pipelines — lint, test, deploy on every push — for **zero-downtime releases**."

 **Risk flag:** the 40% has no source story yet. Fill Drill 2, or say "*noticeably faster*" instead.

Drill 1 — "Walk me through your pipeline stages."

Strong answer (adjust to your real pipeline):

"**One — install and lint:** Composer and npm install with caching, then linters. **Two — tests:** PHPUnit and pytest against a test database; if any test fails, the pipeline stops, so nothing broken reaches production. **Three — build:** build the Docker image, tag it with the **commit SHA**, push it to a registry."

Four — deploy: [FILL: staging automatically, production on merge / after approval]. **Five — after deploy:** run migrations, then a **health check**; if it fails, roll back to the previous image."

Drill 2 — "How did you measure 40% faster?"

 **Pick your real version:**

- **A — Pipeline time:** "Total time from merge to live went from about [FILL: X] minutes to [FILL: Y] minutes — mainly because of [FILL: dependency caching / Docker layer caching / parallel tests / no manual steps]."
- **B — Manual → automated:** "Before, deploys were partly manual — SSH, pull, install, restart — and took about [FILL: X] minutes of an engineer's time. The pipeline cut that by about 40% and removed human mistakes."
- **C — Honest:** "It's an estimate from comparing typical deploy times before and after. I'd say noticeably faster — around 40%."

Drill 3 — "How do you roll back?"

Strong answer:

"Because every image is tagged with the commit SHA, rollback means **redeploying the previous image** — no rebuild. The tricky part is the **database**: a rollback doesn't undo a migration. So migrations are **backward-compatible** — for example, add a new column first, deploy code that uses it, and remove the old column only in a later release. That way the old code still works if we roll back."

Bonus drills

Question	Short strong answer
"Docker vs a VM?"	"A VM virtualizes a whole machine with its own OS. A container shares the host kernel and packages only the app and its dependencies — lighter, and it starts in seconds."
"How do you make images small and fast to build?"	" Multi-stage builds — build tools in one stage, only the runtime in the final image. Put rarely-changing layers, like dependencies, first so they're cached."
"Where do secrets go?"	"Never in the image or the repo. Environment variables injected at deploy time from the CI secret store or [FILL: AWS Secrets Manager / a server-side .env]."
"How do you get zero-downtime deploys?"	"Start the new container, check its health, switch traffic, then stop the old one. [FILL: Nginx reload / rolling restart / load balancer]."
"Kubernetes?"	"I'm familiar with it — pods, deployments, services — but I haven't run it in production." (<i>CV says familiar — be honest.</i>)

Trade-off line

- **Main:** "Running the full test suite on every push makes the pipeline slower, but it's much cheaper than finding a bug in production."

- **Alt:** "Docker adds a build step and some learning, but everyone runs exactly the same environment — that ended our 'works on my machine' problems."

Numbers to know

Number	Source	How to defend	Safe phrasing
~40% faster deployments	CV 	Drill 2	"noticeably faster, about 40%"
No environment-mismatch incidents	CV	Same image everywhere	 if true
Deploy time before → after	[FILL]	—	"from about X to Y minutes"

Key phrases

Phrase	When to use	Example
"works on my machine"	Why Docker	"Docker ended our 'works on my machine' problems."
"tagged with the commit SHA"	Rollback	"Every image is tagged with the commit SHA, so rollback is fast."
"backward-compatible migrations"	Rollback	"We only ship backward-compatible migrations."
"the pipeline is the gate"	Tests	"Nothing reaches production unless the pipeline is green."

C10 Short answers (30–60 s each)

Keep these **short**. If they want more, they'll ask. Each one has a likely follow-up.

"How do you use AI in your work?" (S8)

"I use **Claude Code every day** — for code generation, review, debugging and docs. I set up a custom workflow with **separate agents** for database, backend, frontend and test work, and **project rules** they must follow. But I **review every diff myself**, and tests are a required phase, so AI makes me faster without lowering quality."

- *Follow-up: "How do you catch AI mistakes?"* → "I read every change like a PR from a new teammate. Tests must pass, and I check the risky areas myself — tenant isolation, security, anything touching money. [FILL: one real example of catching an AI mistake.]"

"Do you practice TDD?"

"For new **service and API logic**, yes — I write the test first with **PHPUnit or pytest**, see it fail, then write the code. Tests are a **required phase** of every feature, so regressions get caught in **CI**, not in production. For UI work I'm more pragmatic — I test the logic, not every pixel."

- *Follow-up: "Unit vs integration tests?"* → "Unit tests check one piece in isolation and are fast. Integration tests check pieces working together — like an API endpoint with a real test database. For APIs I write mostly **feature/integration tests**, because that's where real bugs hide."

"Tell me about Am Coders."

"My first role. I **designed, built and shipped a multi-tenant SaaS** sold on the **CodeCanyon** marketplace — RBAC, a REST API, a Vue frontend, and a full **subscription lifecycle**: trial, renewal, cancellation, with HMAC-verified payment webhooks. I handled buyer bug reports directly and usually fixed issues **within 24 hours**. I started as an intern and was **promoted to Software Engineer within one month**."

- *Follow-up: "What did supporting real buyers teach you?"* → "To write clear error messages and docs — every unclear message became a support ticket. And to reproduce the buyer's exact problem before fixing it."

"Tell me about SoftArch." (beyond GirlyShopper)

"Besides GirlyShopper, I built a **grocery delivery platform** with multi-store inventory and advance orders, and tuned MySQL for peak order volumes. I also **managed Linux VPS servers** — Ubuntu and Debian — for several production apps, and set up **Dockerized CI/CD** for zero-downtime releases."

"Your CV mentions research."

"I have **two papers under review**: emotion detection from customer reviews using **LLMs and NLP**, and a **statistical analysis** of suicide case patterns in Bangladesh. It taught me to work carefully with data, question my assumptions, and explain results clearly."

- *Follow-up: "What model did you use?"* → [FILL: model names, dataset size, result — only what's true]

"You list NestJS / Kubernetes / Go. How strong are you?"

Be honest — this builds trust: "**NestJS** — I'm actively learning it; it's close to what I know from Express and Laravel's structure. **Kubernetes** — I'm familiar with the concepts, but I haven't run it in production. **Go** — I'm learning it now, because I'm interested in systems and reliability work."

- *Follow-up: "What have you built in Go?"* → [FILL: e.g. small practice programs] — never claim production work.

"Why are you leaving Sokrio?" (not on the CV, but it will come up)

"I've learned a lot at Sokrio — I grew from building features to leading architecture work and on-call. Now I want to work on a **bigger platform with a larger engineering team**, where I can learn from senior engineers and work at a bigger scale. Field Nation's field-workforce marketplace is very close to what I built, so it feels like the right next step." [FILL: adjust to your real reason — keep it positive, never criticize your current company]

Master checklist — every [FILL] in this doc

Fill these **before** the live mock. Each one is a question the interviewer might really ask.

#	Block	What to fill	Priority
1	C1 S	The real pain of the monolith	●
2	C1 D1	What the SOP service does (1 sentence)	●
3	C1 D2	Data sync during the parallel run (pick A / B / C)	●
4	C1 D3	How the 20% was measured + period	●
5	C1 D4	How consumers de-duplicate · retry count	●
6	C1	Team size · duration · your personal part	●
7	C2 S/R	Which report · before → after seconds	●
8	C2 D1–D2	Real EXPLAIN finding · real index columns	●
9	C2 D3	TTL-only or event-based invalidation	●
10	C2 D4	How MongoDB stays in sync with MySQL	●
11	C3	One real incident (what, detect, cause, fix, prevent)	●
12	C3 D2	How the 65% was measured	●
13	C3	Names of the six apps	●
14	C4 S	Why the Vue → React rewrite	●
15	C4 D2	Which metric for +30% engagement	●
16	C4	Team size · how work was split · your part	●
17	C5-A	How the tenant is resolved (domain / header)	●
18	C5-B	Gateway name · "zero failed" period	●
19	C5-C	Atomic update or row lock · how 60% was measured	●
20	C6	One real client integration problem	●
21	C7 D1	Real GPS-spoofing signals · flag or reject	●
22	C7 D2	Your modules' role in the 27%	●
23	C8	One concrete Node.js service you built	●
24	C9	CI tool · how the 40% was measured	●

40-minute practice plan for Part C

Min	Do this
0–10	C1 out loud: model answer (2 min) + all 5 drills. Record yourself on your phone.
10–18	C2 out loud + Drills 1–2 (EXPLAIN + index order).
18–24	C3 : incident story + SLI/SLO/SLA definition (under 30 s).

Min	Do this
24–30	C4 + C7 : leadership story + Why-FN bridge.
30–36	C5 : multi-tenancy (90 s) + payments (90 s) + POS race condition.
36–40	C10 rapid round: 30 s per question, no notes.

After recording, check:

- Did I start with the **direct answer**, or with background?
- Did I say **"we"** when I should say **"I"**? (Say "I" for your own decisions and work.)
- Did I use **past tense** for finished work? ("I **led**", "we **cut** over", "it **dropped**")
- Any number I couldn't explain? → mark it in the checklist above.